

TYPESEC

TYPE-LEVEL SECURITY FOR
AGENTIC AI



ALEXY KHRABROV



Typesec

Type-Level Security for Agentic AI

Alexy Khrabrov

2026-08-06

Contents

1	Preface	2
2	The Problem	3
3	The Security Model	4
4	Architecture at a Glance	5
5	Workspace Tour	8
6	typesec-core	10
6.1	Capabilities	10
6.2	Permissions	11
6.3	Secure Values	11
6.4	Resources	12
6.5	Policy Results	12
6.6	Minting	13
6.7	Typestate	14
6.8	Combinators	14
7	OAuth, Arcade, WorkOS, and Typesec	15
7.1	The Comparison	15
7.2	Where DIDs Fit	16
7.3	DID-Wrapped Prompts and Ollama	17
7.4	DID Examples	18
7.5	What Arcade Does	19
7.6	What WorkOS Does	20
7.7	The Integrative Architecture	21
8	typesec-agent	22
9	typesec-memory — Marciana	23
9.1	Memory is a resource	23
9.2	Labels travel with content	24
9.3	Provenance is part of the security model	25
9.4	Time, graph, and retrieval	25
9.5	The guarded agent surface	26
9.6	The exact v1 boundary	27
10	typesec-integrations	28
11	RBAC	29

12	ODRL	30
13	Macros	31
14	CLI	31
15	Examples	33
15.1	RBAC Agent	33
15.2	ODRL Agent	33
15.3	OAuth Provider Integrations	33
15.4	Company Graph with Grust and Sail	34
15.5	Python LangChain-Style Tool Gating	34
16	Tests and Validation	35
17	What We Improved	36
17.1	Rialto (0.9.0)	37
17.2	Murano (0.10.0)	38
17.3	Burano (0.11.0)	38
17.4	Torcello (0.12.0)	38
17.5	Lido (0.13.0)	38
18	Design Tradeoffs	39
19	Roadmap	40
19.1	Marciana after v1	40
19.2	Broader TypeSec work	40
20	Conclusion	41

1 Preface

This book describes Typesec, a system for carrying authorization evidence in types while leaving policy decisions explicit at runtime. It is both a design record and a codebase guide. The intent is not just to say what the repository contains, but to explain why each piece exists, how the pieces fit together, and what security property the system is trying to make harder to accidentally lose.

One direct inspiration was David Andrzejewski’s Scale By the Bay 2018 talk, “Privacy aware data science in Scala with monads and type level programming”. That talk connected notebook-era data science, information-flow control, and type-level programming. It also pointed back to the Haskell SecLib model of wrapping data in a security-labeled container. Typesec takes that lineage in a Rust direction: runtime policy still decides, but the proof and the protected data shape are carried by types. A cleaned transcript of David’s talk is included at `docs/david-andrzejewski-scale-by-the-bay-2018-transcript.md`.

The project began from a simple observation: agentic software makes ordinary authorization mistakes more dangerous. A human-facing web request usually has a short life. An AI agent can be long-running, can call many tools, can make many intermediate decisions, and can wander through code paths that were not anticipated when the first permission guard was written. If security depends on remembering to call `check_permission()` before every dangerous action, the system is only as strong as its most forgetful call site.

Typesec moves the central permission proof into Rust's type system. Runtime policy still matters. RBAC and ODRL policies are parsed and evaluated at runtime. But once a policy engine allows an action, the result is not just a boolean. The result is an unforgeable typed value:

```
Capability<CanWrite, Report>
```

That value becomes the proof required by sensitive APIs. Code that writes a report should accept a write capability. Code that reads sensitive data should accept a sensitive-read capability. Code that exfiltrates data should be forced to say so in its type signature. The ideal is a codebase where the compiler can show us the security boundary before the program ever runs.

The newer `SecureValue<L, T, R>` layer extends that idea from operations to data itself. A sensitive string can be transformed while it remains opaque. It can be combined with other values while the type-level label tracks the more restrictive result. It cannot be unwrapped or declassified unless the caller has the corresponding capability.

The repository now lives at:

```
git@github.com:querygraph/typesec.git
```

The local branch `main` tracks `origin/main`. Before publishing, the workspace was checked with:

```
cargo check --workspace
```

The Grust integration was also checked with:

```
cargo check -p typesec-cli --example company_graph_grust_sail
```

2 The Problem

Most authorization code is guard-based. A function starts with a condition:

```
if acl.check(user, "write", resource) {  
    resource.write(data);  
}
```

This pattern is familiar, easy to understand, and often good enough for small systems. It also has a structural weakness: the guard is separate from the operation. Every call path has to remember the guard. Every refactor has to preserve the guard. Every new integration has to know which guard applies.

Agentic systems stress this weakness. An agent may plan, retrieve data, call a tool, revise the plan, call another tool, ask another model, and then publish a result. Each of those steps can cross a policy boundary. Some boundaries are ordinary application boundaries, such as read versus write. Others are AI-specific boundaries, such as train, infer, or exfiltrate.

The central design question for Typesec is:

Can we make the permission proof part of the API shape, so the dangerous action is not callable unless the proof is already in scope?

Typesec does not remove runtime policy evaluation. It changes what runtime policy evaluation returns. A policy check that allows an action mints a capability. The capability is then consumed or borrowed by APIs that require that access.

The most important consequence is that permission becomes visible in function signatures:

```
fn write_report(cap: &Capability<CanWrite, Report>, report: &Report) {  
    // The body can assume the policy engine approved this subject  
    // for this permission and this resource.  
}
```

There is no matching function that takes only `&Report` and quietly writes it. If such a function appears, it is visible during review as a policy bypass.

3 The Security Model

Typesec is built around a small set of invariants.

First, capabilities are unforgeable outside `typesec-core`. The `Capability<P, R>` fields are private, and the constructor that skips policy evaluation is `pub(crate)`. External crates cannot write:

```
Capability { ... }
```

and they cannot call the unchecked constructor. They must ask a policy engine.

Second, permission types are sealed. The `Permission` trait is implemented by Typesec's own marker types, such as `CanRead`, `CanWrite`, `CanDelete`, `CanReadInternal`, `CanReadSensitive`, `AiCanInfer`, `AiCanTrain`, and `AiCanExfiltrate`. External crates cannot invent `CanDoAnything` and use it as a back door.

Third, resource types implement a `Resource` trait. A resource exposes a stable identifier, and that identifier is what runtime policy engines evaluate. The type parameter still matters. `Capability<CanWrite, Report>` is not the same type as `Capability<CanWrite, CustomerRecord>`, even if both are represented at runtime by strings.

Fourth, agent authentication is represented as `typestate`. The core agent starts as:

```
Agent<Unauthenticated>
```

After successful authentication it becomes:

```
Agent<Authenticated>
```

Only the authenticated state exposes capability-requesting behavior. This makes “forgot to authenticate” a type error instead of a late runtime surprise.

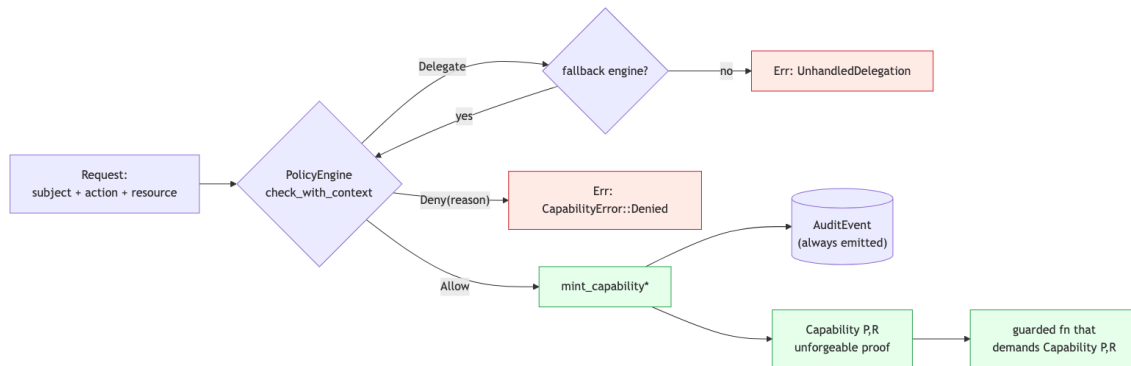
Fifth, every policy decision is auditable. The runtime bridge emits structured events through tracing, so production users can connect the policy layer to their logging or SIEM infrastructure.

These invariants are intentionally modest. Typesec is not pretending that the type system can know every business rule at compile time. The type system enforces that sensitive operations require a proof. Runtime policy decides whether the proof should be minted.

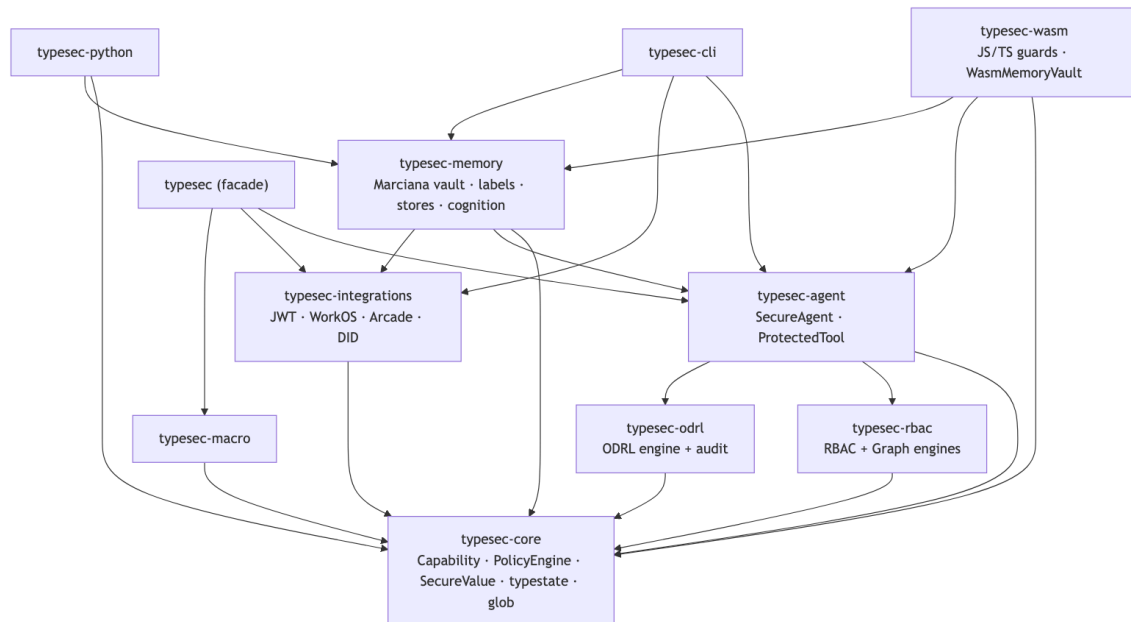
4 Architecture at a Glance

Before the prose tour, five pictures of the load-bearing pieces. The Mermaid sources are inline below (the book build renders them; GitHub renders them too).

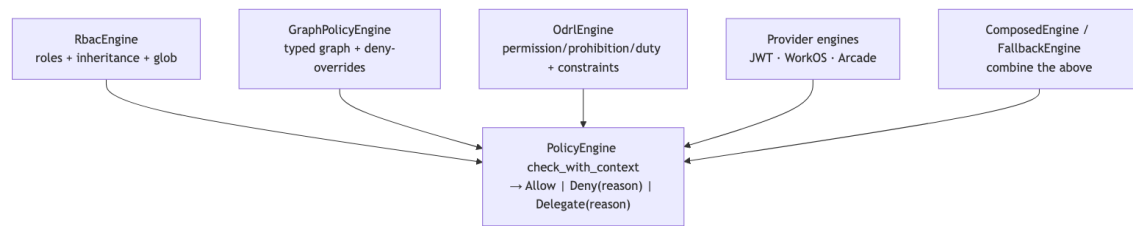
The single most important diagram is the capability-minting flow. A `Capability` is unforgeable proof, and the only production path to one runs a `PolicyEngine` and emits an audit event; a denial is a typed error, never a capability.



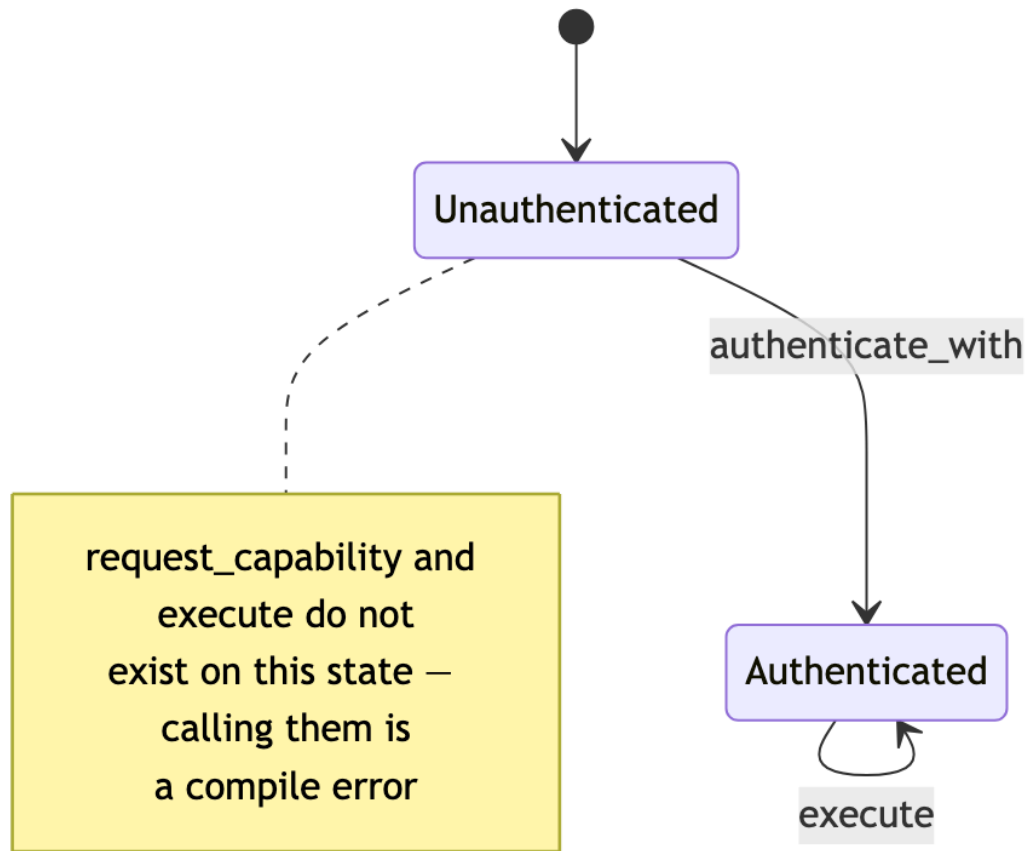
The workspace is eleven crates layered on `typesec-core`. The umbrella `typesec` crate re-exports the core, policy, agent, macro, and integration surfaces behind feature flags; `typesec-memory` and `typesec-wasm` remain purpose-built distribution surfaces rather than being hidden inside the facade.



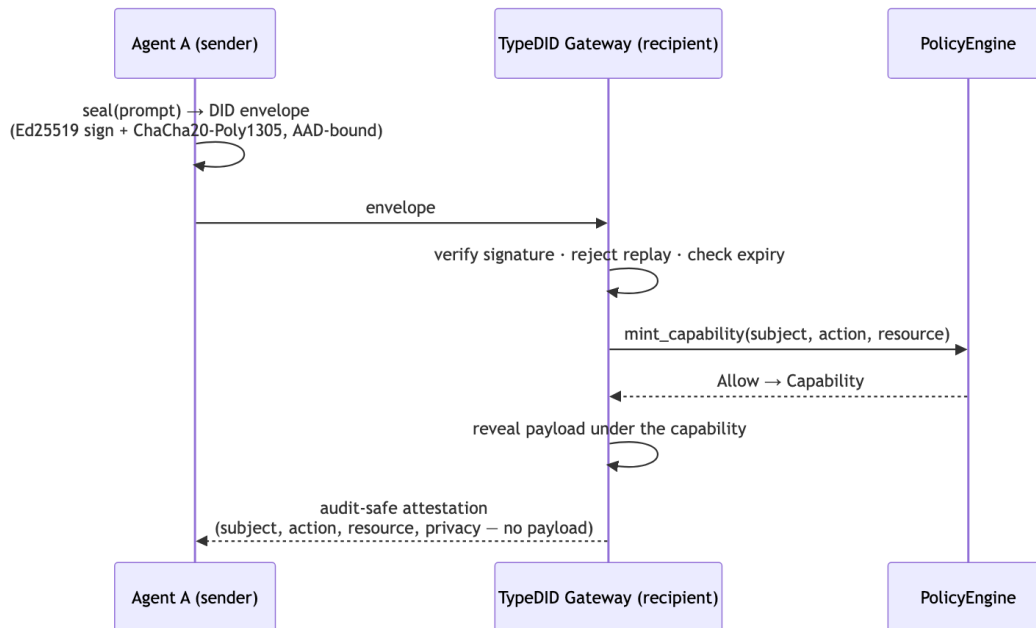
Every engine — RBAC, the graph engine, ODRL, and the provider-backed engines — implements the same `check_with_context` contract returning `Allow`, `Deny`, or `Delegate`, so they compose under `ComposedEngine` and `FallbackEngine`.



SecureAgent<S> is a typestate machine: the capability-requesting and execute methods exist only on the Authenticated state, so calling them on an unauthenticated agent is a compile error, not a runtime check.



When agents collaborate, a prompt is sealed into a DID envelope whose ciphertext is AEAD-bound to its routing and timing identity; the gateway verifies signature, replay, and expiry, then reveals the plaintext only under a typed capability and returns an audit-safe attestation that records who did what to which resource without exposing the payload.



5 Workspace Tour

The repository is a Rust workspace with eleven crates:

<code>typesec</code>	facade crate re-exporting the common API
<code>typesec-core</code>	traits, phantom types, <code>Capability</code> , <code>SecureValue</code> , <code>PolicyEngine</code> ,
<code>typestate</code>	
<code>typesec-rbac</code>	YAML RBAC parser, validator, engine, and code generator
<code>typesec-odrl</code>	YAML ODRL subset, constraints, prohibitions, and audit events
<code>typesec-agent</code>	SecureAgent wrapper for async capability requests and execute
<code>typesec-integrations</code>	JWT/OIDC, WorkOS FGA, and Arcade-style tool auth engines
<code>typesec-macro</code>	derive and policy macros for typed role declarations
<code>typesec-cli</code>	validate, check, generate, and run commands
<code>typesec-memory</code>	Marciana vault, labels, stores, guarded memory tools, and
<code>cognition traits</code>	
<code>typesec-python</code>	PyO3 bindings for Rust-backed Python policy gates
<code>typesec-wasm</code>	JavaScript/TypeScript tool guards and the in-memory WASM vault

The integration layer also includes an initial DID messaging boundary. DIDs are used as verifiable subjects and key-discovery handles, while policy still flows through `PolicyEngine`. A DID-wrapped prompt can be verified, decrypted into a `SecureValue<Secret, String, GenericResource>`, and sent to Ollama only after `Typesec` mints both the sensitive-read and AI-inference capabilities. The local implementation uses `Ed25519DidKeyStore` for Ed25519 signatures, X25519 key agreement, and ChaCha20-Poly1305 payload encryption. A deterministic non-cryptographic demo store remains available only for tests or behind the `demo-crypto` feature; production backends can also plug in `DIDComm/JWE`, Hyperledger Indy VDR, or a Universal Resolver adapter behind the same resolver trait.

The repository also includes:

```
examples/rbac_agent.rs
examples/odrl_agent.rs
examples/provider_integrations.rs
examples/did_messaging.rs
examples/typedid_agent_communications.rs
examples/memory_agent.rs
examples/memory_consolidation.rs
examples/pydantic_ai_capabilities.py
examples/typedid_framework_adapters.py
examples/company_graph/company_graph_grust_sail.rs
examples/company_graph/graph_policy_schema.rs
examples/company_graph/langchain_company_graph.py
examples/company_graph/pydantic_company_graph.py
policies/rbac-example.yaml
policies/odrl-example.yaml
docs/did-messaging.md
docs/typedid-agent-communications.md
docs/typedid-ecosystem.md
docs/company-graph-grust-sail.md
docs/oauth-provider-integrations.md
docs/typesec-and-auth-frameworks.md
tests/python/test_cli_policy.py
tests/python/test_pydantic_ai_capabilities.py
tests/python/test_typedid_framework_adapters.py
```

The root workspace uses Rust 2024. Common dependencies are declared in `[workspace.dependencies]`: `tokio/futures` (`async`), `serde/serde_json` and `serde_yaml` (the last is the maintained `serde_norway` fork, renamed via Cargo's package = so existing `serde_yaml::` paths keep working), `garde` (validation), `clap`, `chrono`, `tracing/tracing-subscriber`, `thiserror/anyhow`, `zeroize`, `syn/quote/proc-macro2`, `glob`, `jsonwebtoken`, `request`, the PyO3 binding (`pyo3`), and the DID envelope cryptography crates (`ed25519-dalek`, `x25519-dalek`, `chacha20poly1305`, `sha2`, `getrandom`).

Grust is published on crates.io (`grust-graph`, `grust-cypher`, and `grust-sail` are at 0.12.0, codename `Lobster`). The workspace pins each dependency with both a version and a local path, so a checkout that has a sibling `../grust` builds against that working copy (the path wins locally), while the version resolves against crates.io when `Typesec` is published or built without the sibling checkout:

```
# workspace Cargo.toml
grust-graph = { version = "0.12.0", path = "../grust/crates/grust", features =
["typed-zod-rs"] }
grust-cypher = { version = "0.12.0", path = "../grust/crates/grust-cypher" }
grust-sail   = { version = "0.12.0", path = "../grust/crates/grust-sail" }
```

To build the Cypher company-graph example purely from crates.io, drop the path keys and keep the version; to develop `Typesec` and Grust together, keep the sibling `../grust` checkout.

The package is named `grust-graph`, while its library is imported as `grust`. The facade re-exports the core graph API, while `grust-sail` exposes the Sail adapter types directly. The `typed-zod-rs` feature lets Typesec route policy YAML and JSON through Zod schemas before typed Rust structs lower into a Grust graph. The same graph schema is then passed to Grust backends with `put_typed_graph`, and `grust-cypher` lets Typesec apply Cypher DDL constraints or execute an authorized Cypher mutation through the same graph store boundary.

6 typesec-core

The core crate is where the security idea becomes concrete. It contains the unforgeable capability type, permission markers, resource abstraction, policy engine trait, typestate agent, policy combinators, and security-labeled values.

6.1 Capabilities

The central type is:

```
pub struct Capability<P: Permission, R: Resource> {
    id: CapabilityId,
    subject: SubjectId,
    resource_id: ResourceId,
    issued_at: SystemTime,
    expires_at: SystemTime,
    revocation: Option<(RevocationEpoch, u64)>,
    revocation_list: Option<Arc<CapabilityRevocationList>>,
    _permission: PhantomData<fn() -> P>,
    _resource: PhantomData<fn() -> R>,
}
```

At runtime, the value stores a unique capability id, the subject, the resource identifier, and the lease: when the capability was minted, when it expires, and optional revocation bindings. The permission and resource types are represented with `PhantomData`. They cost nothing at runtime, but they force the compiler to distinguish a read capability from a write capability.

A capability is a cached policy decision, not a permanent credential. Every consuming API calls `ensure_active()`, which rejects a capability that has outlived its lease, whose `RevocationEpoch` has been bumped since mint, or whose id appears in a `CapabilityRevocationList`. The default lease is five minutes; `MintOptions` lets callers shorten it per risk — a declassification capability warrants seconds, a low-risk read can hold the default.

The constructor is deliberately hidden:

```
pub(crate) fn new_minted(
    subject: impl Into<SubjectId>,
    resource_id: impl Into<ResourceId>,
    issued_at: SystemTime,
    ttl: Duration,
    revocation: Option<RevocationEpoch>,
    revocation_list: Option<Arc<CapabilityRevocationList>>,
) -> Self
```

That one visibility choice carries much of the design. The capability can be minted by code inside `typesec-core`, but not by consumers. Consumers call `mint_capability`, which first asks a `PolicyEngine`.

Capabilities are intentionally not `Clone`. A privilege should not spread through the program by accident. If code needs to share a capability, it passes a reference. This is a small choice, but it aligns the ergonomics with the security model: possession of a capability is meaningful.

6.2 Permissions

Permissions are marker types. The names are runtime strings when a policy engine needs to evaluate a request, but static types when APIs require proof. The project includes ordinary permissions:

```
read
write
delete
execute
delegate
read_sensitive
write_sensitive
declassify
```

It also includes AI-oriented permissions:

```
ai:infer
ai:train
ai:exfiltrate
```

The exfiltration permission is especially important. If an agent can send data outside a trust boundary, that path should not be hidden inside an ordinary “write” operation. A function that requires `Capability<AiCanExfiltrate, CustomerData>` announces the risk in its signature.

The declassification permission is similarly explicit. If code lowers a protected value from `Sensitive` or `Secret` to `Public`, it should not look like an ordinary read. A function that accepts `Capability<CanDeclassify, Report>` is telling reviewers that this path intentionally releases a derived value.

6.3 Secure Values

SecLib’s useful lesson for `Typesec` is that the protected value itself should have a shape the compiler can recognize. `Typesec` now has:

```
pub struct SecureValue<L: PrivacyLevel, T, R: Resource> {
    value: T,
    resource_id: String,
    _label: PhantomData<fn() -> L>,
    _resource: PhantomData<fn() -> R>,
}
```

The fields are private. Consumers cannot extract value by pattern matching. They can transform it:

```
let email: SecureValue<Sensitive, String, CustomerRecord> =
    SecureValue::protect(customer.email, &customer);

let domain = email.map(|addr| addr.split('@').last().unwrap_or("").to_owned());
```

The label is preserved through map. When two protected values are combined with zip, the Join trait computes the more restrictive privacy label:

```
Public + Sensitive -> Sensitive
Sensitive + Secret -> Secret
Internal + Public -> Internal
```

This is deliberately small. It is not a complete information-flow-control language, and it does not model every SecLib operation. But it gives application code a first-class way to keep sensitive data opaque while still doing useful work with it.

Extraction is limited to explicit release paths. SecureValue<Public, T, R> can be unwrapped with into_public. SecureValue<Internal, T, R> can be revealed with a Capability<CanReadInternal, R>, while sensitive and secret data still require Capability<CanReadSensitive, R>. To lower the label, code must hold:

```
Capability<CanDeclassify, R>
```

That makes declassification visible at the call site and in the policy log.

6.4 Resources

Resources implement:

```
pub trait Resource {
    fn resource_id(&self) -> &str;
    fn resource_type() -> &'static str;
}
```

The policy engine sees the identifier. Rust sees the type. This lets Typesec bridge dynamic policy files with typed application code.

For quick CLI and test paths, the core crate also provides generic resources. Examples define domain-specific resources, such as employee nodes, relationships, company graphs, and employee networks.

6.5 Policy Results

All policy engines return:

```
pub enum PolicyResult {
    Allow,
    Deny(String),
    Delegate(DelegationReason),
}

pub struct DelegationReason {
    pub engine: &'static str,
```

```

    pub reason: String,
    pub context: Option<String>,
}

```

Allow mints a capability. Deny returns an error with a reason. Delegate means this engine has no definitive answer and another engine may decide. The delegation reason records which engine abstained, why it abstained, and optionally the surrounding delegation context. ODRL uses delegation naturally: if an ODRL policy has no matching rule, it can defer to RBAC.

The error type for capability acquisition is:

```

pub enum CapabilityError {
    Denied { reason: String },
    UnhandledDelegation,
    EngineError(Box<dyn std::error::Error + Send + Sync>),
}

```

If a single engine delegates and no wrapper handles that delegation, minting fails with `UnhandledDelegation`. The composition layer solves this for deployed combinations, and the structured delegation reason keeps enough provenance for CLI, Python, and audit output to report where the abstention came from.

6.6 Minting

The minting flow is the core runtime bridge:

```

agent.request_capability::<CanWrite, Report>(&report)
-> engine.check(subject, "write", report.resource_id())
-> PolicyResult::Allow
-> Capability::new_minted(subject, resource_id, now, ttl, revocation,
revocation_list)

```

The function that performs this is `mint_capability`. It emits an audit event for every decision and only calls the hidden constructor after an allow.

Two variants give callers control over the lease. `mint_capability_with` takes `MintOptions`, which carries the TTL, an optional `RevocationEpoch`, and an optional `CapabilityRevocationList`:

```

let epoch = RevocationEpoch::new();
let options = MintOptions {
    ttl: Duration::from_secs(30),
    revocation: Some(epoch.clone()),
    ..MintOptions::default()
};
let cap: Capability<CanDeclassify, CustomerRecord> =
    mint_capability_with(&engine, subject, &customer, &options)?;

// Later – policy reload, incident response, governance change:
epoch.revoke_all();
cap.ensure_active(); // Err(CapabilityUseError::Revoked { .. })

```


A `RevocationEpoch` is a cheap, cloneable shared counter. Capabilities record its value at mint; bumping it invalidates every capability minted before the bump, immediately, without waiting out the TTL. This closes the gap between “the policy changed” and “the proof stops working”.

For narrower incident response, bind capabilities to a `CapabilityRevocationList` and revoke the id of exactly the compromised proof:

```
let crl = Arc::new(CapabilityRevocationList::new());
let options = MintOptions::default().with_revocation_list(crl.clone());
let cap: Capability<CanRead, CustomerRecord> =
    mint_capability_with(&engine, subject, &customer, &options)?;

crl.revoke(cap.id());
cap.ensure_active(); // Err(CapabilityUseError::RevokedById { .. })
```

`mint_capability_for_id` accepts the resource id as a plain string for callers that already have a stable identifier. Async variants (`mint_capability_async`, `mint_capability_with_async`, and `mint_capability_for_id_async`) call the async policy surface and then record through the async audit sink path. The minted capability is bound to that id exactly as in the `&R` form; every consumption site still compares ids at use time.

6.7 Typestate

The typestate module defines:

```
Agent<Unauthenticated>
Agent<Authenticated>
```

The state marker trait is sealed. External crates cannot add fake states. The production transition from unauthenticated to authenticated is `authenticate_with`, which consumes the unauthenticated agent, passes the credentials to an `Authenticator`, and binds the agent to the *verified* subject the authenticator returns — never the claimed one. `JwtAuthenticator` in `typesec-integrations` is the reference implementation; it verifies the token against a JWKS endpoint and rejects credentials whose claimed subject does not match the token’s sub claim.

For tests, demos, and deployments where identity is established out of band, `authenticate_unverified` trusts the claimed subject as-is and logs a warning. The honest name is the point: the dangerous path announces itself at every call site.

Credentials carry their bearer secret in a `Token` newtype. `Token` redacts its contents from `Debug` output and implements neither `Display` nor `PartialEq` — printing a credentials struct cannot leak the secret into logs, and equality against a guessed string cannot become a brute-force oracle. The raw secret is read through a single explicit `expose()` call at the verifier boundary.

The point of this crate is not to own identity. The point is to make the authenticated state visible in the type system after identity has been established.

6.8 Combinators

The combinator module lets multiple policy engines be combined. It supports four strategies:

PriorityOrder	first non-delegating answer wins
AllowIfAll	all definitive engines must allow
AllowIfAny	any allow is enough unless all deny or delegate
DenyOverrides	any deny beats any allow

DenyOverrides is the conservative XACML-style choice. PriorityOrder is useful when a more specific policy should get the first chance to decide and fall back to a broader one only on delegation.

7 OAuth, Arcade, WorkOS, and Typesec

Typesec is not an OAuth replacement. That point matters because the modern agent-security stack already has strong identity and delegation systems. OAuth, OIDC, AuthKit, WorkOS, Arcade, and provider-specific consent flows all solve real problems that Typesec should not try to reimplement.

The better architecture is layered:

OAuth/OIDC	proves identity and delegates external authority
WorkOS	models enterprise users, organizations, sessions, RBAC, and FGA
Arcade	manages agent-facing OAuth/tool authorization for SaaS tools
Typesec	converts allowed decisions into typed local capabilities

This comparison is also recorded in docs/typesec-and-auth-frameworks.md. The short version is:

OAuth proves and delegates identity. Typesec turns authorization decisions into compile-time-visible authority inside agent/tool code.

7.1 The Comparison

The systems operate at different layers:

Typesec	code-level enforcement for agent/tool execution
Arcade	MCP/tool runtime and delegated user auth for external services
WorkOS	identity, OAuth apps, RBAC/FGA, enterprise auth infrastructure

Typesec's core primitive is `Capability<P, R>` plus typestate `Agent<Authenticated>`. Arcade's core runtime primitive is a user-specific tool authorization and managed OAuth/API-key token state. WorkOS's primitives are OAuth/OIDC tokens, organization claims, roles, permissions, and FGA access checks.

That gives each system a natural strength:

Use WorkOS when you need enterprise login, SSO, organizations, RBAC, IdP sync, and resource-scoped FGA for application objects.

Use Arcade when an agent needs to act as a user inside external SaaS systems such as Gmail, Slack, GitHub, Jira, or Google Drive.

Use Typesec when local tool code must not run unless the program holds a typed proof that authorization already happened.

Arcade and WorkOS still return runtime facts. A token verifies. A consent flow completes. An authorization API returns `authorized: true`. Those facts are important, but ordinary application code can still ignore them unless every call site is disciplined. Typesec adds the local last mile: the handler is written to require a typed capability, so the call cannot be made by accident.

7.2 Where DIDs Fit

DIDs add another edge identity and messaging pattern. They are useful when an agent, user, service, model gateway, or organization needs a portable cryptographic identifier that can be resolved without depending on one OAuth provider. A DID document can advertise verification methods and service endpoints. A message can be signed by the sender DID and encrypted for the recipient DID.

That does not replace Typesec policy. DID verification answers:

```
Did the sender control did:key:z... when it signed this envelope?
Was the ciphertext encrypted for this local recipient DID?
Which keys and endpoints are advertised by the DID document?
```

Typesec answers a different question:

```
May did:key:z... run ai:infer on prompt/session/123?
May this code reveal a Secret prompt?
May the result be sent outside the current trust boundary?
```

The DID support therefore belongs in `typesec-integrations`, not `typesec-core`. The integration crate verifies and decrypts the message at the edge, then hands the result to the same `PolicyEngine` and `SecureValue` machinery as every other integration.

The current implementation is deliberately modest:

<code>Did</code>	parsed decentralized identifier string
<code>DidDocument</code>	verification methods, key agreement, service endpoints
<code>DidResolver</code>	trait for resolving a DID to a DID document
<code>DidKeyStore</code>	trait for signing, verifying, encrypting, and decrypting
<code>DidEnvelope</code>	encrypted DID-wrapped prompt envelope
<code>DidMessageGateway</code>	verifier/decrypter that returns a protected prompt
<code>DidOllamaClient</code>	Ollama bridge for verified or still-wrapped prompts

`StaticDidResolver` keeps local resolution deterministic, and `Ed25519DidKeyStore` provides the default local cryptographic key store. The optional `DemoDidKeyStore` is non-cryptographic and available only for tests or with the `demo-crypto` feature. Production deployments can implement the same traits with `DIDComm/JWE`, `HPKE`, `HSM/KMS`-backed keys, `Hyperledger Indy VDR`, `did:web`, `did:key`, or a `Universal Resolver` client.

The `Ed25519` store is rotation-aware. `rotate_key(did, key)` adds a new active version, `active_key_version(did)` reports what new envelopes will use, and `document(did)` advertises non-retired verification methods as active or previous. Previous keys keep in-flight envelopes valid until `retire_key(did, version)` removes them from new documents and makes that verification method fail.

7.3 DID-Wrapped Prompts and Ollama

The first concrete DID use case is an encrypted prompt for a local or modified Ollama server. The conservative path keeps Typesec in charge of reveal:

DID envelope arrives

- > require authVersion = typesec.did-envelope-auth.v2
- > DidResolver resolves sender and recipient DID documents
- > DidKeyStore verifies the complete length-framed envelope transcript
- > DidKeyStore decrypts for the local recipient DID
- > DidMessageGateway returns VerifiedDidPrompt
- > prompt is SecureValue<Secret, String, GenericResource>
- > PolicyEngine mints AiCanInfer and CanReadSensitive capabilities
- > DidOllamaClient sends plaintext to Ollama

The important detail is the SecureValue boundary. The decrypted prompt is not returned as an ordinary string. It becomes:

SecureValue<Secret, String, GenericResource>

That means the client must hold a sensitive-read capability to reveal it:

```
let verified = gateway.open_prompt(&envelope)?;

let infer: Capability<AiCanInfer, _> =
  mint_capability(engine, verified.subject().as_str(), verified.resource());
let read: Capability<CanReadSensitive, _> =
  mint_capability(engine, verified.subject().as_str(), verified.resource());

let ollama = DidOllamaClient::new("http://localhost:11434", "llama3.2");
let response = ollama.chat_verified_prompt(verified, &infer, &read)?;
```

The gateway rejects missing, legacy, and unknown authentication versions. Version 2 length-frames and authenticates routing, timing, policy claims, conversation and reply metadata, key id, nonce, and ciphertext. Its verified prompt and TypeDID message types have private fields, so downstream code can read authenticated values through accessors but cannot forge gateway provenance. After signature verification, the DID-message gateway accepts only prompt and reply types and the TypeDID gateway accepts only TypeDID messages; cross-protocol envelopes are rejected before decryption or replay consumption.

When the Ollama reply needs to travel with the same authority context as the prompt, Typesec can bind the assistant message back to the prompt:

```
let reply = ollama.chat_verified_prompt_bound(
  verified,
  gateway_did,
  &resolver,
  &key_store,
  &infer,
  &read,
)?;
```

This returns a new signed and encrypted DID reply envelope. The reply envelope uses a fresh DID-shaped id, keeps the prompt's action, resource, and privacy metadata, and stores a `reply_to` reference containing the prompt envelope id and digest. That reference is included in the reply signature, so the reply cannot be detached from the prompt or rebound to a different prompt without invalidating the envelope.

There is also a compatibility path for a DID-aware Ollama fork:

```
let response = ollama.chat_wrapped_prompt(&envelope)?;
```

That sends the whole envelope under a `did_envelope` JSON field. This is useful when the model gateway expects the DID-wrapped prompt directly. The stricter local path should remain the default for Typesec-controlled applications because the plaintext reveal is guarded by typed capabilities.

7.4 DID Examples

The repository includes a runnable DID example:

```
cargo run -p typesec-cli --example did_messaging
```

It creates two local `did:key` identifiers:

```
let alice_key = Ed25519DidKey::from_seed(b"alice");
let gateway_key = Ed25519DidKey::from_seed(b"typesec-ollama-gateway");

let alice = Did::key(alice_key.signing_public());
let gateway_did = Did::key(gateway_key.signing_public());
```

Then it registers DID documents in a `StaticDidResolver`, creates a `DidEnvelope`, verifies and decrypts that envelope through `DidMessageGateway`, and sends the prompt through `DidOllamaClient` only after policy mints `Capability<AiCanInfer, _>` and `Capability<CanReadSensitive, _>`.

That example is intentionally offline. It uses `Ed25519DidKeyStore`, `StaticDidResolver`, and `RecordingHttpClient`, so it exercises the Typesec boundary without requiring a live Ollama server, a public DID registry, or a Hyperledger ledger.

The same trait shape supports more realistic DID methods:

<code>did:key</code>	derive the document from key material; best for local tests
<code>did:web</code>	fetch a DID document from an HTTPS domain
<code>did:indy</code>	read DID state from Hyperledger Indy through Indy VDR
public DID	call a Universal Resolver and translate the DID document

For Hyperledger development, a local setup has three pieces:

VON Network	local development Indy Node network and ledger browser
Indy VDR proxy	Rust-side ledger reader and DID resolver endpoint
Typesec adapter	<code>DidResolver</code> implementation that maps VDR JSON to <code>DidDocument</code>

The local ledger can be started outside this repo:

```
git clone https://github.com/bcgov/von-network.git
cd von-network
./manage build
REGISTER_NEW_DIDS=True ./manage start
curl -fsS http://localhost:9000/genesis -o /tmp/von-genesis.txn
```

VON's browser is normally available at <http://localhost:9000>. Its README describes it as development-only infrastructure and exposes a /genesis endpoint for clients. With REGISTER_NEW_DIDS=True, the browser can also enable the local “Authenticate a New DID” flow for writing sandbox DIDs through a known trust anchor.

Indy VDR can then read the local ledger:

```
git clone https://github.com/hyperledger-indy/indy-vdr.git
cd indy-vdr
cargo build --bin indy-vdr-proxy
./target/debug/indy-vdr-proxy -p 9001 -g /tmp/von-genesis.txn
```

Smoke tests:

```
curl -fsS http://localhost:9001/genesis
curl -fsS http://localhost:9001/nym/<UNQUALIFIED_DID>
```

Indy VDR also documents a DID resolver endpoint:

```
GET /1.0/identifiers/{DID or DID_URL}
```

The future IndyVdrResolver should call that endpoint for qualified did:indy values, or /nym/<DID> for a simple local VON smoke test, then translate the ledger response into the local DidDocument model. After that, the prompt path is unchanged: DID resolution and cryptography happen at the edge, while Typesec policy still controls reveal and inference.

7.5 What Arcade Does

Arcade is closest to the agent-tool problem. It handles OAuth 2.0, API keys, and user tokens for tools. In an agent setting, that means a user can authorize Gmail once, Arcade can store and refresh the provider tokens, and an agent can later call a Gmail tool on that user's behalf without the model seeing secrets.

Arcade also separates two boundaries:

```
resource-server auth  protects access to the MCP server or gateway
tool-level auth       authorizes the specific third-party tool call
```

That distinction maps cleanly to Typesec. Arcade can answer:

```
Has user@example.com authorized Gmail.ListEmails?
```

Typesec can then mint:

```
Capability<CanExecute, GenericResource>
```

where the resource id is Gmail.ListEmails or a local alias such as gmail/list.

The repository's `ArcadeToolAuthEngine` is intentionally small. It maps a local resource id to an Arcade tool name, asks the configured Arcade endpoint whether the user's authorization is complete, and returns `PolicyResult::Allow` only when the provider response is complete. Pending authorization is a denial with the provider URL included when available.

```
let arcade = ArcadeToolAuthEngine::new(arcade_api_key)
    .with_tool_mapping("gmail/list", "Gmail.ListEmails");

let gmail = GenericResource::new("gmail/list", "tool");
let cap: Capability<CanExecute, GenericResource> =
    agent.request_capability(&gmail).await?;
```

The important design choice is that Arcade remains the OAuth/tool runtime, while Typesec remains the local proof system. Arcade knows how to complete OAuth. Typesec knows how to make the local tool handler require proof.

7.6 What WorkOS Does

WorkOS is broader enterprise auth infrastructure. AuthKit and Connect cover login, OAuth applications, SSO, user sessions, organizations, token issuance, and token verification. WorkOS Fine-Grained Authorization extends RBAC into a hierarchical, resource-scoped model for application objects such as organizations, workspaces, projects, and apps.

For Typesec, the useful WorkOS split is:

JWT claims	fast checks for org-wide roles and permissions
FGA API	precise checks for a specific action on a specific resource

That split fits Typesec's existing policy-composition model. The token can be verified once and converted into a `JwtClaimsEngine`. The JWT engine can allow obvious org-wide permissions and delegate everything else. The WorkOS engine can then call the Authorization API for resource-specific checks.

```
let jwt_engine = Arc::new(JwtClaimsEngine::from_permissions(
    "user@example.com",
    ["org:view".to_string()],
));

let workos_engine = Arc::new(WorkOsFgaEngine::new(workos_api_key));

let engine = PolicyEngineBuilder::new()
    .add_engine(jwt_engine)
    .add_engine(workos_engine)
    .strategy(CombineStrategy::PriorityOrder)
    .build();
```

Typesec resource ids use a simple convention for WorkOS FGA:

```
project/proj_123
workspace/ws_123
```


WorkOsFgaEngine parses the prefix as the resource type slug and the suffix as the external resource id. A write capability request on project/proj_123 becomes a WorkOS-style permission check for project:write on that resource.

7.7 The Integrative Architecture

The typesec-integrations crate puts these ideas behind the same PolicyEngine trait used by RBAC, ODRL, and graph policies.

OIDC/AuthKit access token

- > JwtAuthenticator verifies signature, issuer, audience, expiry
- > VerifiedSubject becomes the authenticated Typesec subject

Capability request

- > JwtClaimsEngine handles fast org-wide permissions
- > WorkOsFgaEngine handles app resource permissions
- > ArcadeToolAuthEngine handles external SaaS tool authorization
- > PolicyResult::Allow
- > Capability<P, R>
- > ProtectedTool<P, R, _> can run

Representative setup from examples/provider_integrations.rs looks like this:

```
let engine = PolicyEngineBuilder::new()
    .add_engine(Arc::new(JwtClaimsEngine::from_permissions(
        "user@example.com",
        ["read".to_string()],
    )))
    .add_engine(Arc::new(WorkOsFgaEngine::new(workos_api_key)))
    .add_engine(Arc::new(
        ArcadeToolAuthEngine::new(arcade_api_key)
        .with_tool_mapping("gmail/list", "Gmail.ListEmails"),
    ))
    .strategy(CombineStrategy::PriorityOrder)
    .build();
```

Then local tool code can be shaped around the capability:

```
fn gmail_list(_resource: &GenericResource) -> ToolFuture<'_> {
    Box::pin(async {
        // Call the actual MCP/Arcade/Gmail implementation here.
        Ok(())
    })
}

let resource = GenericResource::new("gmail/list", "tool");
let tool = ProtectedTool::<CanExecute, _, _>::new(
    "gmail.list",
    "List email messages",
    resource,
    gmail_list,
```

```
);
```

```
let cap: Capability<CanExecute, GenericResource> =  
  agent.request_capability(&GenericResource::new("gmail/list", "tool")).await?;  
  
tool.invoke(&agent, &cap).await?;
```

This is the central integration claim. WorkOS can say who the user is and what app resource they can access. Arcade can say whether the user authorized the external SaaS tool. Typesec can make the local implementation impossible to invoke without carrying that authorization as a typed proof.

8 typesec-agent

typesec-core stays dependency-light. It does not need tokio. The typesec-agent crate wraps the core typestate agent in a higher-level SecureAgent that exposes async capability requests and execution.

The unauthenticated constructor is:

```
let agent = SecureAgent::new(Arc::new(engine));
```

Authentication returns a new type:

```
let agent = agent.authenticate_with(Credentials::new("agent:bot", token),  
&jwt_auth)?;
```

Only SecureAgent<Authenticated> has:

```
request_capability:::<P, R>(&self, resource: &R)  
request_capability_with:::<P, R>(&self, resource: &R, options: MintOptions)  
execute(&self, cap: &Capability<P, R>, resource: &R, action)
```

request_capability awaits the async policy surface exposed by typesec-core. Synchronous engines use the default async adapter, while I/O-bound engines can override the async methods and avoid blocking the executor. The _with variant plumbs MintOptions through, so an agent can request a short-lived or revocation-bound capability without dropping down to the core minting functions.

The execute method captures the project's main ergonomic pattern. It takes a capability reference and a resource reference, logs the execution, and then runs the action. The newer ProtectedTool wrapper applies the same idea to agent-tool and MCP-style handlers: the tool advertises a required permission and resource, and invoke requires the matching Capability<P, R>. A ToolRegistry can hold multiple protected tools, list their specs for discovery, and invoke one by name after downcasting the supplied capability back to the tool's typed proof.

```
let tool = ProtectedTool:::<CanExecute, _, _>::new(  
  "gmail.list",  
  "List Gmail messages",  
  GenericResource::new("gmail/list", "tool"),  
  gmail_list,
```

```
);

tool.invoke(&agent, &execute_cap).await?;

let mut registry = ToolRegistry::new();
registry.register(tool);
let specs = registry.list_specs();
registry.invoke("gmail.list", &agent, &execute_cap).await?;
```

The underlying execute method runs an async closure. The closure cannot be reached through this method without a capability.

9 typesec-memory — Marciana

An agent’s durable memory is not merely a retrieval feature. It accumulates the most sensitive material the agent has encountered, carries assertions across sessions, and can turn one successful prompt injection into a standing source of future instructions. A `user_id` predicate on a vector query is useful scoping, but it does not answer the governing questions: which verified agent may remember, recall, transform, share, or forget this record; for which purpose; at which clearance; and with what evidence afterward?

Lido applies the existing Typesec model to that problem. The subsystem is named **Marciana**, after Venice’s Biblioteca Marciana: a library understood not as a pile of text, but as an institution for provenance, custody, classification, access, and stewardship. Its security core lives in the `typesec-memory` crate. The realized v1 adapter lives in Grust and the authenticated edge in `qg-rust`. The accepted extraction moves the memory-specific adapter and reusable service assembly into standalone Marciana; `qg-rust` and `qg-python` remain consumers. That division keeps authority, persistence, product composition, transport, and framework integration in their proper homes.

9.1 Memory is a resource

Every memory space implements the ordinary Typesec Resource contract:

```
memory/user:alice/profile
memory/agent:planner/procedural
memory/team:research/shared
```

The existing RBAC, graph, and ODRL engines can therefore govern memory without a parallel policy language. Each operation requires a different typed proof:

```
Capability<CanRead, MemorySpace>
Capability<CanWrite, MemorySpace>
Capability<CanDelete, MemorySpace>
```

The proofs are not interchangeable. Write authority does not imply delete authority, and a capability for one space fails against another. They retain the same lease, revocation, attenuation, and audit behavior as every other Typesec capability. A planner can delegate a short-lived read capability to a sub-agent without handing it write or forget authority.

The vault makes that distinction part of its API rather than an instruction in a system prompt:

```

let write: Capability<CanWrite, MemorySpace> =
    mint_capability_for_id(&engine, subject, space.resource_id(), &options)?;

let id = vault.remember(
    &space,
    &write,
    MemoryDraft::new(
        MemoryKind::Profile,
        MemoryContent::text("prefers dark mode"),
        Provenance::Operator,
    ),
)?;

```

MemoryVault checks that the capability is active and bound to the target space. When constructed with `with_policy`, it also rechecks policy at use time, so a still-live proof does not silently ignore a purpose or governance change. The model never receives a constructor for capabilities or an unguarded handle to stored content.

9.2 Labels travel with content

Scope answers whose memory this is. A label answers how sensitive its contents are. Every stored record carries a runtime `Public`, `Internal`, `Sensitive`, or `Secret` tag. Only the vault may rehydrate the private content field, and the typed Rust recall path declares the maximum sensitivity of the destination context:

```

let recalled: Recall<Internal> = vault.recall::<Internal>(
    &space,
    &read,
    RecallQuery::text("display preference"),
    &RequestContext::default().with_purpose("support"),
)?;

```

Records at or below `Internal` arrive as typed hits. Hotter records arrive only as `RedactedHit` metadata. `CanReadSensitive` can reveal through the guarded escalation path up to `Sensitive`; **Secret remains sealed** because `v1` does not model a stronger reveal authority. At JSON and tool-call boundaries, where a runtime string cannot become a Rust type parameter, `recall_at` applies the same ceiling check dynamically.

Purpose is a separate gate. A record limited to support is absent from an analytics recall even when the caller otherwise holds read authority. An expiry becomes enforceable retention state: `reap_expired` requires delete authority, tombstones the records, and emits the same audit trail as an explicit forget. With the receipts feature, deletion can also produce signed evidence for workflows that require offline verification.

Consolidation preserves information-flow labels. Replacing several facts with a summary joins their labels, so a summary of `Sensitive` material is itself at least `Sensitive`; transformation cannot launder it into a cooler context. The entire supersede-and-replace plan is submitted through `MemoryStore::apply_batch` as one unit, which transactional adapters can commit atomically.

9.3 Provenance is part of the security model

Marciana records where a fact came from. A verified TypeDID envelope, a human operator, a guarded tool result, a conversation, and raw model text do not begin with the same trust. Raw model text is born quarantined, and ordinary recall, neighborhood recall, semantic recall, and consolidation exclude quarantined records by default. This is the defense against memory poisoning: an injected sentence may be retained for inspection without silently becoming durable truth that guides a later session.

That guarantee has an important v1 edge. The Rust query model can explicitly request quarantined records, but Marciana does not yet expose a mandatory promotion operation that requires `CanDeclassify`, nor does every consolidation path require proof of promotion before quarantine can be cleared. Hosted use must add that audited promotion boundary and prove quarantine propagation across all derived records. The book treats this as unfinished security work, not as a property inferred from the default query behavior.

9.4 Time, graph, and retrieval

A memory record distinguishes when it was observed from when it was valid. Supersession invalidates an old assertion instead of overwriting history, so “Alice lives in Venice” can remain inspectable as a past belief beside its successor without being returned as current truth. Explicit forgetting is the separate destructive path.

The in-tree `InMemoryStore` makes the contract easy to test and embed. The feature-gated `GrustMemoryStore` adds entity relationships and neighborhood recall. The v1 `querygraph-memory` adapter, currently located in `Grust` and used as the behavior-preserving Marciana extraction source, carries the same `MemoryStore` contract to persistent `Turso/libSQL`-backed `Grust` universal tables. In every case, the graph or semantic index returns candidate record identifiers; the vault decides whether content can be revealed. Retrieval can narrow and rank a candidate set, but it cannot widen authority.

The `SemanticIndex` seam follows the same rule. `Typesec` ships a deterministic `KeywordIndex`, while `QueryGraph` v1 proves an in-process `VectorIndex` with a privacy-aware embedder. The index is not the source of truth and does not return content. `Remembering` feeds it on a best-effort basis, `forgetting` prunes it, and `recall` always returns through the vault’s space, validity, quarantine, and label checks.

Reference cognition is similarly constrained. Extractors and analytics produce `MemoryDrafts` and `ConsolidationPlans`; they do not mutate the store directly. The deterministic `RuleExtractor` and local-model `OllamaExtractor` demonstrate that raw model output remains an untrusted proposal until authorized code sends it through `remember` or `consolidate`. The v1 adapter’s reference analytics are also **plan producers only**. Distributed cognition must preserve that inert proposal boundary: `MemoryVault::apply_cognition` freshly authorizes and validates the proposal before handing an opaque prepared commit to the trusted store transaction. If the transaction committed but its reply was lost, `recover_cognition_outcome` accepts only the exact job and proposal digest, checks a current `CanWrite` capability and policy before lookup, and discloses validated historical evidence without reconstructing a proposal, rerunning mutation authority, or applying

a second write. Lookup absence, conflicts, corruption, and adapter failures deliberately share one unavailable result.

That boundary is deliberately narrow and bounded. Network and model-produced JSON enters through `CognitionProposal::from_json_slice`, which rejects an oversized body before parsing; proposal and nested cognition types reject unknown fields. Application independently resolves the exact job, native algorithm and version, governed scan, immutable snapshot, effective projection, source manifest, verified TypeDID request, subject, and purpose. Its durable idempotency identity contains only the space, job, and an opaque digest of the authority scope. Fixed budgets cover proposal bytes, source records and source IDs, projections, mutations, cloned lineage, evidence, and affected IDs. An `Applied` backend outcome must carry the exact audit the vault just prepared. An `AlreadyApplied` outcome instead discloses an immutable historical commit: its canonical shape, authority-scoped identity, proposal, binding, source, TypeDID, governed-scan, authorization, evidence, and affected IDs must match, while historical preparation time and policy-decision evidence remain those committed by the original transaction. Signed commit receipts apply the same canonical and bounded treatment to the evidence they expose. These are TypeSec security rules; they do not introduce a Cognee runtime, store, adapter, or wire-compatibility surface.

Strict cognition decoding is also a schema boundary, not an invitation to discard data. Recursive unknown-field rejection reaches shared nested domain types such as memory drafts, content, entities, and provenance, which may also appear in stored or cross-service documents. Before deploying the stricter decoder over older data, inventory and validate those documents, transform any extra fields through an explicit versioned migration, and only then advance readers and writers together. Future intentional fields require a schema version and migration; they must not depend on an older reader silently ignoring them.

9.5 The guarded agent surface

Marciana memory operations are normal protected tools:

```
memory.remember  -> write  -> resource from the space argument
memory.recall    -> read   -> resource from the space argument
memory.forget     -> delete -> resource from the space argument
```

`memory_bindings()` registers those mappings with the same deny-by-default `ToolCallGuard` used for OpenAI, Anthropic, LangChain, Pydantic AI, and MCP. `MemoryToolRouter` validates the normalized request again, mints the operation-specific capability through the configured policy engine, and only then calls the vault. The CLI exposes the path through `typesec memory-serve`; Python has `MemoryGate`; and `typesec-wasm` provides `WasmMemoryVault` for the in-memory web boundary.

The realized `QueryGraph` service preserves the same sequence across a network:



qg-python keeps the Ed25519 signing seed in typed runtime dependencies. qg-rust accepts the verified signing DID—not a caller-supplied JSON subject—as the policy principal. The signature binds sender, recipient, action, exact route, and body digest. Authentication failures return 401; an authenticated subject denied by TypeSec receives a 403 receipt.

The executable proof remembers a governed result as one authorized specialist, terminates and restarts qg-rust, recalls the durable record as a differently credentialed supervisor, and denies a validly signed outsider. This is a real restart-persistence and identity-bound authorization proof, not a claim that the service is already a hosted multi-tenant product.

9.6 The exact v1 boundary

Marciana v1 is complete at the durable local-service boundary. It includes the vault, runtime labels, clearance-typed recall, redaction and guarded reveal, purpose filtering, retention, tombstones and receipts, provenance quarantine, bi-temporal supersession, guarded agent tools, Python and WASM surfaces, Grust reference storage, a versioned backend conformance corpus, transactional consolidation, reference extraction, semantic-index hooks, persistent Turso, identity-bound qg-rust routes, and the Pydantic AI demonstration.

The following limits are equally part of the contract:

- `MemoryId::next()` is monotonic only within one process. A restarted process that writes into an existing durable store can collide with an older id until the durable-id contract is upgraded.
- TypeDID signatures bind identity, route, action, recipient, and body, but v1 does not durably claim a replay nonce across replicas. Durable cross-replica replay protection remains post-v1.
- The Turso/Grust adapter pushes only memory-space equality into the backend. The shared conformance-pinned matcher applies the remaining query dimensions after retrieval.
- QueryGraph's VectorIndex is in-process and nonpersistent. Persistent, tenant-scoped LanceDB ANN is future work; ANN remains a ranking aid rather than an authorization mechanism.
- Reference extraction and analytics produce inert plans only. There are no distributed Sail cognition jobs in v1.
- Tenant isolation is enforced by TypeSec at the vault boundary. V1 has no hosted multi-tenant control plane, physical isolation tiers, quotas, migrations, backup/restore program, or SLO-backed service.
- The v1 querygraph-memory RELATES representation identifies an edge by its structural (`from`, `label`, `to`) shape. It cannot preserve two assertions that differ only by `fact_id`; explicit assertion identity and fuller lineage are post-v1 schema work.
- Quarantine exclusion is the safe default, but mandatory audited CanDeclassify promotion and quarantine propagation through every derived record are not complete.
- CanReadSensitive reveals through Sensitive; Secret remains sealed.

These limits are not reasons to weaken the v1 claim. They make the claim reviewable: Lido delivers governed, persistent agent memory with a tested end-to-end proof, while durable identifiers, replica-safe replay, persistent ANN, assertion-level lineage, distributed cognition, and hosted operations remain named work with explicit acceptance gates.

10 typesec-integrations

The integrations crate is intentionally outside typesec-core. Provider adapters need HTTP clients, JWT validation, provider-specific endpoints, and credential handling. The core crate should not own those dependencies.

The crate currently contains six modules:

http	small injectable JSON HTTP client abstraction
jwt	OIDC/JWT verification and JWT-claims policy engine
workos	WorkOS FGA policy engine
arcade	Arcade-style tool authorization policy engine
pydantic_ai	capability metadata for Pydantic AI v2 tools
did	DID documents, encrypted prompt envelopes, and Ollama bridge

The http module provides `ReqwestHttpClient` for production use plus `StaticHttpClient` and `RecordingHttpClient` for tests and examples. This lets the example exercise provider-like behavior without real credentials:

```
let workos_http = StaticHttpClient::new().with_response(
    "https://api.workos.test/authorization/organization_memberships/user@example.com/check",
    json!({ "authorized": true }),
);
```

The JWT module has two layers. `JwtAuthenticator` validates a token against a JWKS endpoint, issuer, audience, algorithm list, and expiry. Once a token is verified, `VerifiedSubject` carries the subject, organization id, membership id, role, and permissions. `JwtClaimsEngine` then acts as a fast policy engine for permissions already embedded in the token.

The WorkOS module translates a Typesec resource id into a WorkOS FGA check. For example:

```
resource: project/proj_123
action:   write
check:    project:write on project/proj_123
```

The Arcade module handles external tool authorization. It maps local resource ids to Arcade tool names and only allows execution when the provider reports a completed authorization:

```
let arcade = ArcadeToolAuthEngine::new(arcade_api_key)
    .with_tool_mapping("gmail/list", "Gmail.ListEmails");
```

The DID module handles decentralized identifiers and encrypted prompt envelopes. It exposes a resolver trait and a key-store trait so real DID methods and production cryptography can be added without changing the Typesec capability path:

DidResolver	-> DidDocument
DidKeyStore	-> verify/decrypt envelope
Gateway	-> SecureValue<Secret, String, GenericResource>
PolicyEngine	-> Capability<AiCanInfer, _> and Capability<CanReadSensitive, _>
Ollama	-> receives plaintext only after typed authority exists

TypeDID extends that DID boundary from prompt-only messages to agent communications. TypeDidProfile negotiates the secure envelope profile, TypeDidConversation binds the outer task, session, or room id plus protocol, mode, and profile into the envelope signature, and TypeDidGateway opens encrypted opaque payload bytes as SecureValue<Secret, Vec<u8>, GenericResource>. Transport adapters such as A2aTypeDidAdapter, AcpTypeDidAdapter, BandSecureEnvelopeAdapter, and HttpTypeDidAdapter keep A2A, ACP, BAND, and HTTPS responsible for their own lifecycle while TypeDID owns cryptographic sender/recipient binding and the Typesec policy handoff.

The same boundary applies to Python agent frameworks. LangChain and Pydantic AI should not reimplement DID cryptography or become policy engines. Instead, a Rust TypeDidGateway or future native Python binding should produce a verified message view, and framework adapters should use typesec check --json or the native Python gate to decide whether a tool may see the payload. The examples/typedid_framework_adapters.py example shows this shape without depending on either framework: a LangChain-style middleware wrapper and a Pydantic-style dependency object both gate the verified TypeDID message before tool invocation.

These engines are not special cases in the capability system. They implement the same PolicyEngine trait as RBAC, ODRL, and graph policies. That is the point: external OAuth and enterprise authorization decisions become ordinary inputs to mint_capability.

This pattern is small enough to be adopted by application code directly. A database write wrapper can require a write capability. A vector-store retrieval wrapper can require a read capability. An outbound HTTP publisher can require an exfiltration capability.

The crate also includes AgentBuilder, which can wrap a single engine or build a composed engine from a primary and fallback. The default composed behavior is priority order: the primary decides unless it delegates.

11 RBAC

The typesec-rbac crate implements role-based access control from YAML. A policy has roles and assignments:

```
roles:
  - name: analyst
    permissions: [read, read_sensitive]
    resources: ["reports/*", "metrics/*"]
  - name: admin
    inherits: [analyst]
    permissions: [delete, delegate]
    resources: ["*"]

assignments:
  - subject: "agent:data-pipeline"
    roles: [analyst]
```

The engine compiles this into subject grants. Role inheritance is flattened during construction, not recomputed from scratch at every check. A request then checks:

subject -> assigned roles -> effective grants -> permission and glob match

The glob matching is intentionally simple. Resource patterns such as reports/*, employee/**, and * are enough for the first version. The code uses the glob crate rather than ad hoc string prefixes.

RBAC is the system's practical baseline. It answers common operational questions:

Can agent:data-pipeline read reports/ql?

Can agent:deploy-bot write code/main.rs?

Can agent:superuser delete anything?

The crate also contains code generation support. typesec generate can emit typed Rust structs from an RBAC policy. This is one of the paths toward the larger promise: if a role is renamed in policy, code referring to the old typed role should fail to compile.

12 ODRL

The typesec-odrl crate implements a focused subset of the Open Digital Rights Language. ODRL is useful for AI agents because it can express obligations, prohibitions, and contextual constraints.

A policy can say:

```
policies:
- uid: "policy:ai-agent-001"
  type: Set
  rules:
  - type: permission
    assignee: "agent:summarizer"
    action: read
    target: "asset:customer-data"
    constraints:
    - leftOperand: purpose
      operator: eq
      rightOperand: "analytics"
  - type: prohibition
    assignee: "agent:summarizer"
    action: exfiltrate
    target: "asset:customer-data"
```

This says the summarizer can read customer data for analytics, but cannot exfiltrate it. That distinction matters for agents because the same underlying data may be safe to summarize internally and unsafe to send outside the system.

The ODRL engine evaluates subject, action, target, and constraints. Constraints are evaluated against a ConstraintContext, which can include purpose, date-time, and custom key values. The engine indexes rules at construction by assignee and action, with a same-assignee fallback for

ODRL use wildcard rules; target globs and constraints are still evaluated at decision time. The CLI exposes --purpose for the common case.

Conflict resolution is conservative:

prohibition beats permission

If a matching prohibition passes its constraints, the result is a deny. If a permission matches and no prohibition overrides it, the result is allow. If no rule applies, the result is delegate.

The crate also emits structured ODRL audit events. These include the policy UID, matched rule type, subject, action, target, verdict, and constraint evaluation results. This gives operators a reason trail, not just a final boolean.

13 Macros

The typesec-macro crate sketches two developer-experience paths.

The first is a derive macro:

```
#[derive(TypesecRole)]
#[role(permissions = "read,write", resources = "code/*")]
pub struct Engineer;
```

The second is an inline policy macro:

```
policy! {
  role Analyst {
    can [read, read_sensitive] on ["reports/*", "metrics/*"];
  }
  role LeadAnalyst extends Analyst {
    can [write] on ["reports/drafts/*"];
  }
}
```

extends flattens inherited permissions and resource patterns at macro expansion time, matching the RBAC YAML model while keeping the generated Role impls as simple static slices.

Macros are not the security core. The core is capabilities and policy-engine minting. The macros are there to make typed policy declarations less tedious and to give future generated code a natural shape.

14 CLI

The CLI is the bridge for humans, scripts, and non-Rust agents. It has four commands:

validate	parse and validate a policy YAML file
check	evaluate one subject/action/resource query
generate	emit typed Rust code from an RBAC policy
run	simulate agent execution under policy

The check command is especially important because it makes Typesec usable from Python or shell without native bindings:

```
cargo run -q -p typesec-cli -- check \
  --policy policies/rbac-example.yaml \
  --subject agent:data-pipeline \
  --action read \
  --resource reports/q1
```

An allow exits successfully and prints an allow result. A deny exits with status 1. A delegate exits with status 2. That makes the command a simple policy oracle for external tools.

For agent wrappers that should not parse human output, the same check can emit JSON:

```
cargo run -q -p typesec-cli -- check \
  --policy policies/rbac-example.yaml \
  --subject agent:data-pipeline \
  --action read \
  --resource reports/q1 \
  --json
```

That prints a stable decision object with fields such as `decision`, `allowed`, `subject`, `action`, `resource`, and the detected format, while preserving the same exit codes. This closes the earlier machine-readable CLI gap and makes `typesec check --json` the recommended boundary for Python and shell agents.

The format is detected from YAML shape unless `--format rbac` or `--format odrl` is passed explicitly. ODRL checks can receive a purpose:

```
cargo run -q -p typesec-cli -- check \
  --policy policies/odrl-example.yaml \
  --subject agent:summarizer \
  --action read \
  --resource customer-data \
  --purpose analytics
```

The Python tests exercise both the exit-code boundary and the JSON boundary, so the CLI contract is checked from the same subprocess seam that non-Rust agents use.

`typesec run` can also execute a multi-agent scenario file:

```
scenario:
  name: rbac smoke
  policy: ./policies/rbac-example.yaml
  format: rbac
  steps:
    - agent: agent:data-pipeline
      action: read
      resource: reports/q1
      expect: allow
    - agent: agent:data-pipeline
      action: write
      resource: reports/q1
      expect: deny
```

Run it with:

```
cargo run -q -p typesec-cli -- run --scenario scenario.yaml
```

The trace authenticates each listed agent with the local simulation token, checks the requested action/resource pair, and prints whether each optional expect value matched the actual allow, deny, or delegate result. Any expectation mismatch makes the command exit nonzero after printing the trace.

15 Examples

Typesec includes examples at several layers: pure Rust RBAC, pure Rust ODRL, OAuth-provider integration, and company-graph integrations that show how an agent tool boundary might look in a larger system.

15.1 RBAC Agent

The RBAC example shows an authenticated agent requesting a capability and then executing a protected action. The interesting part is not the business object. The interesting part is the call shape: the protected operation is not called until after policy has minted a capability.

The example also demonstrates denial. An agent assigned to a role with read permissions cannot use a write permission unless the policy grants it.

The updated RBAC example also demonstrates the SecLib-inspired data container. It protects a report summary as `SecureValue<Sensitive, String, GenericResource>`, maps it to a length digest while it remains opaque, denies declassification for the ordinary analyst, and then allows a `privacy_reviewer` role to declassify the digest after policy mints `Capability<CanDeclassify, GenericResource>`.

15.2 ODRL Agent

The ODRL example demonstrates contextual policy. Purpose matters. A read for analytics may be allowed while a read for billing delegates or denies. An exfiltration action can be prohibited even if a different action on the same target is allowed.

15.3 OAuth Provider Integrations

The provider integration example is the concrete demonstration of the Arcade and WorkOS architecture described above. It does not require live provider credentials. Instead, it uses `StaticHttpClient` to stand in for WorkOS and Arcade responses while still exercising the same policy-engine and capability-minting path.

Run it with:

```
cargo run -p typesec-cli --example provider_integrations
```

The example composes three engines:

```
let engine = PolicyEngineBuilder::new()
    .add_engine.jwt_claims()
    .add_engine.workos()
```

```

.add_engine(arcade)
.strategy(CombineStrategy::PriorityOrder)
.build();

```

The flow demonstrates three grants:

```

read org/acme                allowed from JWT claims
write project/proj_123       delegated to mocked WorkOS FGA
execute Gmail.ListEmails     delegated to mocked Arcade tool auth

```

The final step invokes a `ProtectedTool<CanExecute, _, _>`. That is the key piece of the example: after the provider engines allow the tool execution, Typesec still requires the local tool implementation to receive the typed execute capability before it can run.

15.4 Company Graph with Grust and Sail

The company graph example is the most complete integration. It models a company hierarchy:

```

employee:evelyn CEO
  <- REPORTS_TO employee:priya VP Engineering
    <- REPORTS_TO employee:marco Engineering Manager
      <- REPORTS_TO employee:nia Senior Software Engineer
        <- REPORTS_TO employee:omar Data Engineer

```

The policy assigns different graph-writing powers to different agents:

```

agent:executive-chief  writes executive data, company graph, and sensitive
network
agent:hr-onboarding     writes non-executive employees and reporting lines
agent:employee-nia      writes only Nia's public self-service profile

```

The Rust example uses the grust facade to build a backend-neutral property graph and to write the graph through the Sail adapter when a Sail SparkConnect server is available at 127.0.0.1:50051. If Sail is not running, the example still demonstrates the Typesec decisions and prints the graph.

The import shape is:

```

use grust::prelude::*;
use grust_sail::{SailConfig, SailGraphStore};

```

The prelude pulls in the core graph types; the Sail adapter types (`SailConfig`, `SailGraphStore`) come from the separate `grust-sail` crate.

15.5 Python LangChain-Style Tool Gating

The Python example is intentionally self-contained. It does not require LangChain to be installed and it does not call an LLM. Instead, it uses the shape of a LangChain tool wrapper:

```

tool call requested
-> TypesecGate.allowed(subject, action, resource)
-> cargo run -q -p typesec-cli -- check ...
-> only then mutate the graph

```

This is a realistic integration boundary. Python agents can ask the Rust CLI for a policy decision before calling a tool. The Python code treats exit code 0 as allow and any nonzero exit code as blocked. A denied tool call raises `PermissionError` before the graph mutates.

This is not as strong as Rust's type-level API, because Python cannot enforce the same phantom-type proof. But it gives Python systems a practical, testable gate today.

16 Tests and Validation

The repository has tests at several levels.

Core unit tests check that capabilities carry the right subject and resource, that read and write capability types differ, and that denied engines do not mint capabilities.

Typestate tests check that authentication transitions from unauthenticated to authenticated and that empty subjects fail.

Agent tests check the full flow:

```
construct agent
authenticate
request capability
execute with capability
```

RBAC tests cover role grants, inheritance, unknown subjects, and denials. ODRL tests cover allowed purpose, wrong purpose, prohibition, and delegation for unknown subjects.

Integration tests in `crates/typesec-agent/tests/integration.rs` exercise the agent layer across more realistic scenarios. Provider integration tests in `crates/typesec-integrations/tests/provider_composition.rs` check the public WorkOS, Arcade, JWT, and capability-composition path. Python smoke tests in `tests/python/test_cli_policy.py` exercise the CLI as a policy oracle.

Marciana adds a security-focused matrix of its own. The audited `typesec-memory` all-features suite includes unit, integration, compile-fail, graph-integration, and doctest coverage. Compile-fail cases prove that external code cannot rehydrate a stored record or recall without the required capability. Runtime cases cover space binding, clearance and redaction, Secret sealing, purpose filtering, quarantine defaults, label joins, transactional consolidation, retention, forgetting, graph neighborhoods, semantic ranking, guarded tool routing, and proposal-free response-loss recovery with anti-oracle and tamper checks. The versioned conformance corpus is run against both `InMemoryStore` and `GrustMemoryStore` in tree.

The cross-repository v1 proof tests a second boundary: `querygraph-memory` passes the same corpus on persistent Turso/Grust storage; `qg-rust` tests exact TypeDID route/body/identity binding, authorization denial, and close/reopen persistence; and the `qg-python` Pydantic AI demonstration proves specialist write, process restart, supervisor recall, signed-outsider 403, and unsigned caller 401 without requiring a provider API key. These tests establish the durable local-service claim. They do not stand in for the post-v1 replica, persistent-ANN, assertion-lineage, or hosted-operations gates.

Benchmark and fuzz tooling cover the hot paths and parser boundaries:

```
cargo bench -p typesec-core --bench policy_check
cargo bench -p typesec-rbac --bench rbac_check
cargo bench -p typesec-odrl --bench odrl_check
cargo fuzz run rbac_yaml -- -max_total_time=300
cargo fuzz run odrl_yaml -- -max_total_time=300
```

The workspace-level verification surface remains:

```
cargo check --workspace
cargo test --workspace
cargo check -p typesec-cli --example provider_integrations
cargo run -q -p typesec-cli --example provider_integrations
```

The Grust/Sail example has its own focused check:

```
cargo check -p typesec-cli --example company_graph_grust_sail
```

The release process runs these checks together with all-feature, Python, WASM, memory-conformance, and book validators before a tag or package publication.

17 What We Improved

The archived improvement notes in `docs/completed/improvements.md` record a useful snapshot of the early engineering work.

The workspace was upgraded to Rust 2024. Compiler failures were fixed across lattice tests, proc-macro parsing, examples, and doctests. Clippy was made to pass across all targets by tightening APIs, deriving defaults, documenting public ODRL fields, and applying simplifications.

The audit integration test was stabilized with a global capture subscriber for the test binary. That avoids a racy thread-local subscriber and makes the audit story more credible.

Python smoke tests were added around `typesec check`, giving non-Rust agent wrappers a low-friction test path.

The company graph examples were added to show Typesec at an application boundary, not just inside small unit tests. The Rust version proves the typed capability story with Grust. The Python version proves that the CLI can gate side-effecting tools in a LangChain-like shape.

The provider integration layer was added to show how Typesec fits under OAuth-based systems instead of replacing them. `JwtAuthenticator`, `JwtClaimsEngine`, `WorkOsFgaEngine`, and `ArcadeToolAuthEngine` all feed the same `PolicyEngine` boundary. `ProtectedTool` then demonstrates the local last mile: provider authorization is not merely observed; it becomes a typed capability required by the handler.

The new comparison document, `docs/typesec-and-auth-frameworks.md`, explains the strategic position: WorkOS handles enterprise identity and resource authorization, Arcade handles agent-oriented SaaS tool authorization, and Typesec makes the resulting local authority impossible to forget at the call site.

The Grust dependency was bumped to the 0.12 line (codename Lobster). Grust is published on crates.io, and the workspace pins each Grust crate with both a version and a local path (`../grust`): a checkout with a sibling Grust working copy builds against it, while the published version resolves for crates.io builds and when Typesec itself is published — see the Workspace Tour.

A security review pass then hardened the runtime half of the system to match what the type-level half advertises. `SecureValue` stopped deriving `Debug` and `PartialEq`, so protected data cannot leak through logging or serve as an equality oracle. Capabilities became instance-bound at every consumption site: `reveal`, `declassify`, and `execute` compare the capability’s resource id and subject against the value or agent actually in hand. The typestate transition gained a real trust root through the `Authenticator` trait, with the unverified path renamed to say what it is. The demo DID cryptography moved behind a `demo-crypto` feature, replaced in production by an Ed25519 key store with X25519 agreement and ChaCha20-Poly1305 sealing. Capabilities became short-lived leases.

A follow-up pass finished the remaining items. Capabilities gained configurable TTLs through `MintOptions` and mid-lease revocation through `RevocationEpoch`. Bearer secrets moved into the `Token` newtype, which redacts itself from `Debug` the same way `SecureValue` does. The `async` agent now awaits the `async` policy surface; synchronous engines keep the default adapter, and I/O-bound engines can override it to avoid blocking the executor. The unmaintained `serde_yaml` dependency was replaced by the API-compatible `serde_norway` fork via a package rename, and `thiserror` moved to major version 2.

Marciana then carried the same impossible-to-forget authorization pattern into durable agent memory. The work added a separately versioned crate and conformance contract rather than embedding one preferred database in `typesec-core`: the vault owns capability, label, purpose, quarantine, and forgetting semantics; Grust and `QueryGraph` own durable graph storage and retrieval; `qg-rust` owns the authenticated service edge; and `qg-python` owns the agent-facing capability objects. The end-to-end proof now survives a service restart and denies a differently authorized subject without exposing signing material to model context.

17.1 Rialto (0.9.0)

The Rialto release — the first of a Venetian-landmark release line — is a workspace-wide pass for human reviewability and a round of security and correctness hardening. Every Rust source file was brought under roughly four hundred lines: the 2,635-line DID module became an eleven-file `did/` directory, the policy engine, combinators, secure values, and the graph-policy engine were split along their natural seams, and every unit-test module moved into its own file. Duplicated logic was consolidated — eight combinator strategy functions became one accumulator, four role-inheritance traversals became one walker, and the `WorkOS` and `Arcade` providers now share a single HTTP shell.

On the security side, DID envelopes now sign the kid and nonce, the gateway rejects replays and implausibly future-dated envelopes, the negotiated payload cap is enforced, and the encrypted payload is bound to its envelope identity as ChaCha20-Poly1305 associated data. The ODRL audit

trail was completed: a rule that matches but fails a constraint now emits a `ConstraintFailed` event instead of vanishing, and every matched permission is recorded. The CLI run command now reflects its decision in the exit code, and a GitHub Actions pipeline runs formatting, clippy, tests, and a benchmark smoke step on every change.

17.2 Murano (0.10.0)

The Murano release — the second Venetian landmark in the line — tracks the Grust **0.11.0 (Crab)** dependency. The upgrade is source-compatible: the workspace builds and the full test suite passes with no changes to the typesec crates, so Murano is a dependency-tracking and version-bump release. The codename pool and the release-cutting checklist now live in `RELEASES.md`.

17.3 Burano (0.11.0)

The Burano release — the third Venetian landmark — is a workspace-wide quality, DRY, and test-coverage pass (the codebase was already free of monolithic files), adding roughly forty tests. Glob matching, previously reimplemented three times across the RBAC, graph, and ODRL engines with subtly different wildcard semantics, was unified into a single `typesec_core::glob::GlobPattern`; ODRL now compiles each rule's target once at load rather than on every check. This carries one **behavior change**: a single `*` in a subject, resource, or target pattern no longer crosses `/` separators, so `reports/*` grants `reports/q1` but not `reports/2024/q1` (use `reports/**`) — the safer authorization default, and the semantics the documentation already described. The graph engine now cross-checks every loaded policy graph against the declarative company schema using Grust 0.11's `GraphSchema::validate_graph`, so the schema and the graph can no longer silently drift. JWT verification was hardened to never seed its validator from the token's own `alg` header, and the public error types (`CapabilityError`, `AgentError`, `TaskError`) are now nameable by callers.

17.4 Torcello (0.12.0)

Torcello made Typesec an interoperability layer for agent tools. The deny-by-default guard normalizes OpenAI, Anthropic, LangChain, Pydantic AI, and MCP calls; the CLI adds MCP and streaming-proxy gateways; signed decision receipts, JSON-Schema arguments, OpenTelemetry audit spans, policy-aware tool listing, replayable decision logs, `#[typesec_tool]`, lease attenuation, and conversation typestate carry the same authority contract across Rust, Python, and WASM. Torcello also moves the graph substrate to Grust **0.12.0, Lobster**.

17.5 Lido (0.13.0)

Lido is the Marciana release. It adds `typesec-memory` as workspace member eleven and treats memory as an ordinary governed resource with separate read, write, and delete capabilities. The shipped v1 includes clearance-typed recall, redacted hits, guarded reveal through Sensitive, sealed Secret content, purpose and retention policy, provenance quarantine, bi-temporal supersession, audited forgetting, transactional consolidation, guarded memory tools, Python/WASM/MCP surfaces, Grust reference storage, semantic-index and cognition traits, and a shared conformance suite.

The coordinated QueryGraph proof adds persistent Turso/Grust storage, identity-bound qg-rust routes, and Pydantic AI credentials plus memory across a real restart. Lido does not relabel that proof as a hosted product: durable ids, cross-replica replay, persistent ANN, assertion-level lineage, mandatory declassification promotion, distributed cognition, and hosted multi-tenant operations remain post-v1 work.

18 Design Tradeoffs

Typesec deliberately separates runtime policy from compile-time proof. This is a tradeoff.

Putting all policy in types would be too rigid. Real policies come from YAML, databases, admin consoles, customer configuration, and external governance systems. They change without recompiling the application.

Leaving all policy at runtime is too easy to bypass. A forgotten guard can become a production vulnerability.

Typesec's compromise is:

```
runtime policy decides
typed capability proves the decision happened
protected APIs require the proof
```

Another tradeoff is that capabilities prove a decision at mint time. They now carry a short lease, and capabilities minted with a RevocationEpoch can be invalidated immediately when governance changes — `revoke_all()` kills every outstanding proof minted before the bump. Capabilities can also be bound to a `CapabilityRevocationList` for exact single-proof revocation. What remains unmodeled is *distributed* revocation: the epoch is an in-process counter, so a fleet of agents sharing one policy service still needs a propagation story (a shared CRL, a policy-version claim, a shared epoch service, or capability re-validation against the engine) before revocation is truly global.

The CLI is one of two Python boundaries, and choosing between them is a pragmatic tradeoff. Rust code gets the strongest type-level story. Python code can either shell out to `typesec check --json` (a subprocess oracle: weaker, but trivial to sandbox, inspect, and test) or link the in-process `typesec-python` PyO3 extension (`typesec_native`), which is a built workspace crate, not a hypothetical — see the Macros/CLI and Python example sections.

The Grust integration also records a naming tradeoff. The package is named `grust-graph`, but it exposes the facade library as `grust`, so examples can keep the natural use `grust::prelude::*` shape.

The graph-policy example now uses two validation boundaries. Zod validates author-facing YAML and JSON before the policy graph is built. Grust's `GraphSchema` validates backend writes through `put_typed_graph`, so the same `Agent`, `Role`, `Employee`, `HAS_ROLE`, and `REPORTS_TO` model protects both policy loading and graph persistence.

The OAuth provider integration records a different tradeoff. Typesec should not be a token vault, a hosted IdP, or a full MCP runtime. The integration crate therefore uses small provider-specific

policy engines and injectable HTTP clients. That keeps provider decisions at the edge while preserving the core invariant: only an allow through PolicyEngine can mint the capability that local code requires.

The new SecureValue API is another compromise. It brings information-flow style labeled data into the Rust workspace without pretending to be a full language-level IFC system. The type-level Join lattice is intentionally small and sealed. That keeps the model reviewable, while still giving examples a real opaque value that can be transformed before explicit reveal or declassification.

19 Roadmap

Lido closes the Marciana v1 milestone rather than moving it forward as an ever-receding roadmap item. The non-regression rules remain: every backend still goes through the vault, every optimization returns candidate ids rather than content, every cognitive worker returns an inert proposal, and no hosted layer weakens capability, purpose, clearance, quarantine, or deletion semantics.

19.1 Marciana after v1

Marciana is now an accepted standalone project in the QueryGraph stack, with its extraction in progress. That project owns the native remember, recall, improve, and forget product; durable jobs and recovery; memory-specific Grust, Sail, and LakeCat adapters; wire compatibility; and service operations. Its design and compatibility registry are the active roadmap.

The local Marciana checkout is initialized, but the history-preserving querygraph-memory transplant and qq-rust switch are not complete. Grust's generic cognition, live-Sail, job, outbox, and guarded-commit substrate is being finalized in its owning repository. A local Sail fix for Delta MERGE non-null constraints is not yet a remotely reachable supported pin. These distinctions prevent an in-progress local stack from being presented as a Marciana release.

TypeSec retains the security protocol: capabilities and current policy gate every operation; the vault is the only protected-content and mutation authority; workers emit inert proposals; prepared commits cross a narrow trusted-store boundary; and response-loss recovery discloses immutable evidence without becoming mutation authority. New Marciana capabilities that belong to this trust kernel are implemented and verified here, then consumed through a released version or exact compatible revision. Cognee remains design inspiration only, not a runtime, store, adapter, or API compatibility target.

19.2 Broader TypeSec work

Outside memory, the same direction continues: propagate revocation across a fleet instead of relying on an in-process epoch; version the machine-readable decision schema; compile generated policy types into ordinary downstream examples; deepen DID rotation, resolver, DIDComm/JWE or HPKE, and KMS/HSM support; and add opt-in live provider smoke tests while retaining deterministic mocked CI. Those changes should remain in their owning crates rather than turning typesec-core into a provider, database, or transport framework.

20 Conclusion

TypeSec is a small system with a sharp idea: authorization should not only be a runtime answer. For agentic AI systems, authorization should also leave a typed trace in the program. If a function can write, read sensitive data, train a model, or exfiltrate content, that power should be visible in the function's inputs.

The repository now contains the first coherent version of that idea:

- typed capabilities
- opaque secure values
- sealed permissions
- resource abstractions
- typestate authentication
- RBAC policy evaluation
- ODRL contextual policy and prohibitions
- policy combinators
- async secure agents
- OAuth/OIDC provider integrations
- WorkOS FGA and Arcade-style tool auth engines
- DID-wrapped encrypted prompt handling
- CLI policy checks
- Rust examples
- Python tool-gating example
- Grust/Sail graph integration
- capability-secured Marciana memory spaces
- clearance-typed recall and redacted hits
- purpose, retention, quarantine, and audited forgetting
- transactional temporal memory on the Grust boundary
- guarded memory tools across Rust, Python, WASM, MCP, and Pydantic AI
- identity-bound durable QueryGraph memory proof
- tests and documentation

The design is not finished, but its boundaries are now concrete. TypeSec owns the law: typed authority, labels, purpose, quarantine, and evidence. Grust owns generic graph and guarded-commit persistence, Sail owns distributed compute, and LakeCat owns governed source proof. Marciana owns the memory product, cognition, and memory-specific adapters. QueryGraph consumes Marciana; qg-rust is the current authenticated edge pending router extraction. Agent frameworks receive ergonomic capabilities without receiving private identity material or a bypass around the vault.

That is the arc from Torcello to Lido. Torcello made authorization impossible to forget when an agent calls a tool. Lido applies the same rule to what the agent remembers after the tool call is over. Remembering is useful; governed memory is infrastructure.